

GitHub Actions

CI/CD from first principles

Every concept, taught on a real pipeline: a self-hosted runner builds our app, pushes to the local registry, and updates the chart Argo CD deploys.

WORKFLOW

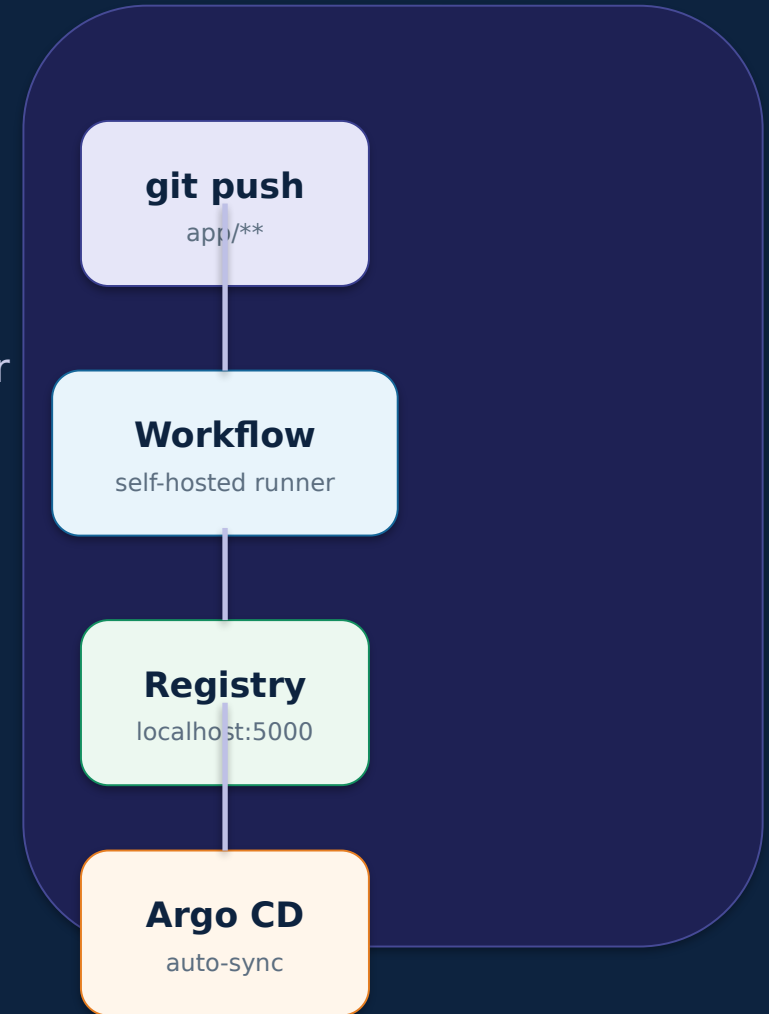
JOB

STEP

RUNNER

ACTIONS

EVENTS



What you will be able to do

Read and write a GitHub Actions workflow file from a blank page.

Explain workflow / job / step / action / runner / event, and how they nest.

Choose GitHub-hosted vs. self-hosted runners, and know why we needed the latter.

Use secrets, the built-in GITHUB_TOKEN, and step outputs safely.

Recognise (and prevent) a CI pipeline that triggers an infinite loop on itself.

Hands-on path

1. Concepts + YAML anatomy
2. Runners (hosted vs. self-hosted)
3. Secrets, tokens, outputs
4. Our real pipeline, end to end
5. Security checklist

Every snippet from here on is copied from this stage's own verified workflow - nothing invented.

What CI/CD actually means

Continuous Integration (CI)

Every change is built and checked automatically, the moment it's pushed - not manually, not at release time. Catches breakage in minutes, not weeks.

Continuous Delivery / Deployment (CD)

The verified result (an image, a package) is automatically made ready to run - and in "Deployment", automatically rolled out. This is what Argo CD (Stage 12) does.

Where this stage sits

GitHub Actions is the CI half: it reacts to a push, builds the app, and pushes an image. It then hands off to Argo CD (already deployed and self-healing since Stage 12) for the CD half. Neither tool does the other's job - that separation is deliberate, not a limitation.

Where GitHub Actions fits

Tool	Runs where	Config lives
GitHub Actions	GitHub-hosted VMs, or your own runners	.github/workflows/*.yml, in the repo
Jenkins	A server you install and operate	Jenkinsfile, or the Jenkins UI
GitLab CI	GitLab-hosted or self-hosted runners	.gitlab-ci.yml, in the repo
CircleCI	CircleCI-hosted, or self-hosted	.circleci/config.yml, in the repo

Same core idea everywhere: describe the pipeline as code, next to the code it builds. We use GitHub Actions here because the app's source already lives on GitHub (Stage 12).

Six words that describe everything

Workflow

One YAML file. The whole automated process.

Job

A group of steps. Runs on one runner.

Step

One action or one shell command.

Action

Reusable packaged step (e.g. actions/checkout).

Runner

The machine that actually executes the job.

Event

What triggered the workflow (push, PR, schedule...).

Nesting: a Workflow contains Jobs; a Job contains Steps; a Step either runs an Action or a shell command.

Anatomy of a workflow file

```
name: Capstone build, push, and gitops bump

on:
  push:
    branches: [main]

permissions:
  contents: write

jobs:
  build-push-bump:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - run: docker build -t img .
```

name:

Cosmetic - what you see in the Actions tab.

on: / permissions:

The EVENT and the token's rights.

jobs: → steps:

Everything actually nests here.

File location

.github/workflows/*.yml - GitHub finds it there automatically.

Triggers: on: — what starts a workflow

```
on:
  push:                # a commit lands on a branch
    branches: [main]
  pull_request:        # a PR is opened / updated
    branches: [main]
  schedule:            # cron - runs on a timer
    - cron: '0 2 * * *'
  workflow_dispatch: {} # a human clicks "Run
                        workflow"
```

A workflow can listen for several events at once - GitHub runs it whenever any of them happens.

push

The one this stage uses: fires as soon as a commit reaches main.

pull_request

Good for running tests/lint BEFORE a merge - doesn't push anything.

schedule / workflow_dispatch

Timer-based, or a manual "Run" button - no code change needed.

Narrowing a trigger: path and branch filters

```
on:
  push:
    branches: [main]      # only this branch
    paths:
      - 'app/**'          # only when these files
changed
```

Why this matters here - a question worth sitting with

This stage's own workflow ends by committing a changed file (chart/capstone/values.yaml) back to the same repo. If the workflow triggered on every push, that commit would trigger another run - which commits again - forever. `paths: [app/**]` means only a change under `app/` starts the workflow; the `chart-bump` commit never matches, so it never re-fires. We will see the real fix, and the real filter, on slide 17.

Jobs run on a runner — hosted or self-hosted

	GitHub-hosted	Self-hosted (this stage)
Where	A fresh VM in GitHub's cloud	A container on our own VM
Lifetime	Created per job, destroyed after	Persistent, we manage it
Network access	Public internet only	Can reach localhost:5000 (ours)
Who maintains it	GitHub	Us
runs-on: value	ubuntu-latest, windows-latest, ...	self-hosted (+ custom labels)

Default choice: GitHub-hosted - zero maintenance. We needed self-hosted for exactly one reason - next slide.

Steps: uses: (an Action) vs. run: (a shell command)

```
steps:
  - name: Checkout                # uses: - someone else's
    packaged step
    uses: actions/checkout@v4

  - name: Compute image tag      # run: - your own shell
    command(s)
    id: tag
    run: echo "tag=${GITHUB_SHA:0:7}" >> "$GITHUB_OUTPUT"
```

actions/checkout is almost always the first step - without it there's no working copy of the repo to build.

uses:

Runs a published, versioned Action - reusable, maintained by someone else (or your org). Pin a version (@v4).

run:

Executes raw shell on the runner - full control, your own script, nothing to trust but the runner itself.

Why THIS pipeline needs a self-hosted runner

GitHub-hosted runner

GitHub cloud VM
ephemeral

**X cannot reach it -
it's bound to
127.0.0.1 on OUR
VM,
never published**

localhost:5000
our registry

Self-hosted runner (ours)

Runner
container, on lab-g2-vm2

**OK same VM,
same network -
"localhost" means
the same machine**

localhost:5000
our registry

The rule of thumb: pick self-hosted when a job needs something only your own network/hardware has (a private registry, GPUs, on-prem services).

Our self-hosted runner: a Docker container

```
FROM ubuntu:24.04
RUN apt-get install -y docker.io curl git jq # Docker CLI, not a daemon
RUN curl -fsSL .../yq_linux_amd64 -o /usr/local/bin/yq
RUN curl -fsSL -o actions-runner.tar.gz \
  "https://github.com/actions/runner/releases/.../actions-runner-linux-x64-2.337.0.tar.gz"
RUN tar xzf actions-runner.tar.gz
ENTRYPOINT ["/entrypoint.sh"] # config.sh, then run.sh
```

Docker-outside-of-Docker

The mounted socket means docker build/push inside this container are really run by the VM's own Docker daemon.

--network host

Matches how a command typed directly on the VM reaches localhost:5000.

```
docker run -d --name capstone-gha-runner --restart unless-stopped \
  --network host --group-add "$(stat -c '%g' /var/run/docker.sock)" \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v capstone-runner-data:/home/runner \
  -e RUNNER_TOKEN="$RUNNER_TOKEN" capstone-gha-runner:local
```

The built-in GITHUB_TOKEN, and permissions:

```
permissions:  
  contents: write  # lets a later step `git push` back to this  
  repo  
  
steps:  
  - run: |  
      git commit -m "ci: bump capstone images to ${TAG}"  
      git push origin HEAD:main  # uses GITHUB_TOKEN  
  automatically
```

actions/checkout reads this token and wires it into git automatically - no manual auth step needed for the push-back.

Auto-generated, auto-expired

GitHub mints GITHUB_TOKEN fresh for every run and revokes it when the job ends - nothing to rotate, nothing to leak long-term.

Least privilege

Default is read-only. This stage explicitly opts into contents: write - and nothing broader.

Secrets vs. plain environment variables

```
env:  
  REGISTRY: localhost:5000      # not sensitive - fine in  
plain env:  
  
steps:  
  - run: docker login \  
    -u ${{ secrets.REGISTRY_USER }} \  
    -p ${{ secrets.REGISTRY_PASS }} # sensitive - from  
Settings > Secrets
```

This stage's registry has no login, so it needed no secret - but that's the pattern for the day it does.

env:

Plain text, visible in the workflow file and in logs. For config that isn't sensitive.

secrets.*

Stored encrypted in repo Settings, injected at run time, automatically masked (***) if it ever hits a log line.

Passing data between steps: outputs

```
- name: Compute image tag (short SHA)
  id: tag                                # name this step
  run: echo "tag=${GITHUB_SHA:0:7}" >> "$GITHUB_OUTPUT"

- name: Build api image
  run: docker build -t "$REGISTRY/capstone-api:${{ steps.tag.outputs.tag }}" app/api
```

A later step reaches back with `steps.<id>.outputs.<name>` - that's how one step's computed value (here, a 7-char git SHA) becomes every later step's image tag.

GITHUB_OUTPUT is a file path the runner gives each step; anything appended to it as key=value becomes that step's output.

Scaling up: matrix, caching, and artifacts

Matrix builds

strategy:
matrix:
 version: [3.11, 3.12]
Runs the SAME job once per
combination, in parallel.

Caching

uses: actions/cache@v4
with:
 path: ~/.cache/pip
 key: \${ { hashFiles(...) } }
Skips re-downloading unchanged
dependencies.

Artifacts

uses: actions/upload-artifact@v4
with:
 path: test-report.html
Hands a file from one job to
the run's summary, or to another
job.

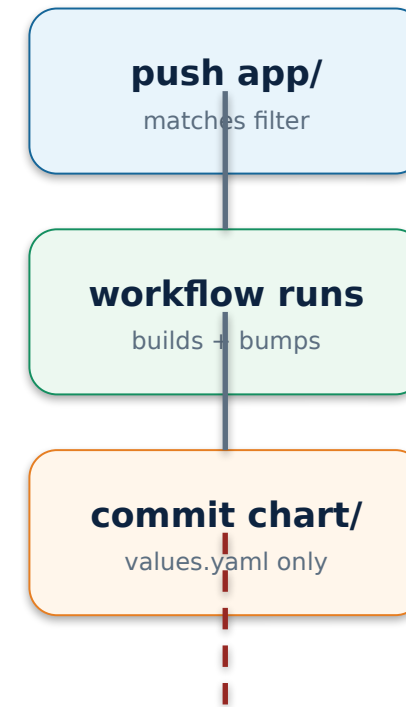
None of these three were needed for this stage's pipeline (one job, no dependency install, nothing to hand off) - but they're the next thing you reach for once a workflow grows.

The loop we actually had to prevent

```
on:
  push:
    branches: [main]
    paths: ['app/**']           # (1) only an app/ change
                                # starts a run
  concurrency:
    group: capstone-image-bump  # (2) one bump commit at a
    time, in order
    cancel-in-progress: false
```

Verified, not assumed

After a real run: `docker logs capstone-gha-runner | grep -c "Running job" -> 1`. Exactly one run per app change, confirmed live - see OUTPUT.md.



does NOT match
app/** → no re-trigger

The real pipeline, end to end

```
jobs:
  build-push-bump:
    runs-on: self-hosted
    steps:
      - uses: actions/checkout@v4
      - run: echo "tag=${GITHUB_SHA:0:7}" >> "$GITHUB_OUTPUT"           # 1. tag
      - run: docker build -t $REGISTRY/capstone-api:$TAG app/api        # 2. build
      - run: docker push $REGISTRY/capstone-api:$TAG                   # 3. push
      - run: yq -i '.api.image.tag = "$TAG"' chart/capstone/values.yaml # 4. bump
      - run: git commit -am "ci: bump..." && git push origin HEAD:main # 5. push
```

5 steps, ~1 minute, on a runner container on this VM. From here, Argo CD (Stage 12, already watching this repo) takes over - it syncs the new values.yaml and rolls api/web to the new tag with zero manual kubectl/helm.

Security checklist before this goes near production

Pin actions to a version (@v4) or, better, a commit SHA - a tag can be moved.

Grant the narrowest permissions: block needed contents: write, not permissions: write-all.

Never echo a secret - GitHub masks known secrets in logs, but only the ones it knows about.

Self-hosted runners on a PUBLIC repo are risky: a fork's PR can run arbitrary code on your machine.

Protect main: require review before a workflow file itself can change.

Review third-party actions like any other dependency - they run with your permissions.

Recap: what you now know

Vocabulary

Workflow > Job > Step; uses: (Action) or run: (shell);
runs-on: picks the runner.

Triggers

on: push/pull_request/schedule/workflow_dispatch;
paths:/branches: narrow it.

Runners

GitHub-hosted by default; self-hosted when a job
needs YOUR network (our registry).

Data & safety

steps.<id>.outputs.*, secrets.*, GITHUB_TOKEN +
permissions:, and paths: to stop a self-triggering loop.

Next: run it yourself

CAPSTONE/Stage13-CI-CD-GitHub-Actions/LAB-MANUAL.md - build the runner image, register it, push a change under app/, and watch the whole loop close.